

## CLOUD EMAIL ANALYZER: SERVERLESS EMAIL THREAT DETECTION WITH AWS, ML, AND CLAMAV

Authors – Andreescu Ovidiu-Ștefan<sup>1</sup>, Anghela Andrei<sup>1</sup>, Sîrbu-Boeți Eduard-Cristian<sup>1</sup>

Scientific Coordinator – Prof. dr. ing. Andrei VASILĂȚEANU<sup>2</sup>

**Abstract:** *Cloud Email Analyzer is an AWS-oriented email security platform for cloud-native message triage. The system receives inbound mail with SES for domains whose MX records are managed in Route53, preserves raw MIME objects in S3, normalizes processing through a canonical `MailReceived` event, routes events with EventBridge, orchestrates Lambda stages with Step Functions, applies TF-IDF-based phishing classification, scans attachments with ClamAV, stores ledger state and artifact pointers in DynamoDB, and exposes verdicts through an authenticated review dashboard.*

**Keywords:** email security, AWS, event-driven architecture, serverless workflow, phishing classification, attachment scanning, DynamoDB

**Rezumat:** *Cloud Email Analyzer este o platformă de securitate e-mail orientată spre AWS, destinată triajului mesajelor suspecte. Sistemul primește mesaje prin SES pentru domenii cu înregistrări MX gestionate în Route53, păstrează obiectele MIME brute în S3, normalizează procesarea prin evenimentul `MailReceived`, rutează evenimentele cu EventBridge, orchestrează etapele Lambda prin Step Functions, clasifică phishing-ul prin TF-IDF, scanează atașamentele cu ClamAV, stochează starea în DynamoDB și afișează verdictele într-un tablou de bord autentificat.*

**Cuvinte cheie:** securitate e-mail, AWS, arhitectură orientată pe evenimente, workflow serverless, clasificare phishing, scanare atașamente, DynamoDB

<sup>1</sup> Students, Faculty of Engineering in Foreign Languages (FILS), IOT 1241EC, National University of Science and Technology POLITEHNICA Bucharest.

<sup>2</sup> Department of Engineering in Foreign Languages (DILS), National University of Science and Technology POLITEHNICA Bucharest.

## 1 Introduction

Email security systems have to treat an inbound message as more than a visible subject line and body. The raw message may contain MIME parts, alternative HTML and plain-text bodies, envelope recipients that differ from the displayed headers, embedded URLs, file attachments, authentication hints, and security verdicts produced by other systems. A useful analysis platform must therefore preserve the original message while extracting evidence into a form that can be queried, audited, and reviewed by users with different privileges.

Cloud Email Analyzer addresses this operational problem through a staged, event-driven AWS architecture. The system receives raw email with Amazon SES for domains whose MX records are managed in Route53, stores the MIME source in S3, parses headers and bodies, extracts attachments, runs a phishing classifier on message text, scans files with ClamAV, persists a review ledger in DynamoDB, and presents the result in a FastAPI and React dashboard. LocalStack is used as a local development tool for exercising AWS-shaped service boundaries without changing the cloud design; the deployed design remains AWS-oriented.

The central design decision is the canonical MailReceived event. After inbound mail is accepted and the MIME object is stored, later stages operate on an event containing tenant, message, envelope, header, verdict, and storage-pointer metadata. That boundary prevents MIME analysis, detector execution, and dashboard review from depending directly on the mechanics of SMTP receipt. It also gives the system a stable audit point: every derived artifact, detector output, and user-visible verdict can be traced back to the same message identifier.

The contribution of the project is an integrated email-triage workflow rather than a new detector in isolation. The phishing classifier is a practical supervised text model, and the attachment scanner uses an established antivirus engine. The project value lies in connecting these pieces with deterministic storage, staged orchestration, authorization, administrator review, and audit logging so that an inbound message becomes reviewable security evidence.

The remainder of the paper is organized as follows. Section 2 positions the project against related work. Section 3 summarizes the system goals and requirements. Section 4 presents the event-driven AWS architecture and the canonical event. Section 5 details the processing pipeline and persistent data model. Section 6 describes the classifier, attachment scanner, and final-verdict rules. Section 7 explains the user and administrator dashboard. Section 8 reports evaluation results from the ML artifacts and implementation checks. Section 9 states limitations and future work. Section 10 concludes the paper.

## 2 Related Work and Positioning

Machine-learning-based phishing email detection has a long research history. Fette, Sadeh, and Tomasic introduced PILFER, a supervised detector based on deception-oriented features, and reported high phishing detection with very low legitimate-mail misclassification on their evaluation set [1]. Later surveys explain why the problem is still not solved by a single classifier: phishing email filtering combines URL, header, content, reputation, and machine-learning signals, while adaptation and evasion remain persistent difficulties [2,3]. Recent comparative work continues this detector-centric direction by evaluating TF-IDF, Word2Vec, BERT, and traditional classifiers for phishing email classification [4].

Cloud Email Analyzer uses that line of work as one layer rather than as the whole system. Its classifier is a TF-IDF-based supervised model, appropriate for efficient text scoring inside a workflow. The architectural question is different from the one answered by detector-only papers: after a detector produces a score, the platform still has to preserve the raw message, store derived

artifacts, scan attachments, aggregate heterogeneous verdicts, enforce user-specific access, and keep an audit trail.

D-Fence is closer to this project because it presents a multi-module phishing email detection system that combines structure, text, and URL analysis for enterprise-scale phishing detection [5]. Cloud Email Analyzer takes a complementary route. Instead of optimizing a new detector ensemble, it emphasizes cloud ingestion, persistent evidence, review ergonomics, and operational traceability around detector outputs. The comparison is useful because it separates two dimensions of email security work: how accurately a detector recognizes malicious intent, and how the surrounding platform turns detector evidence into controlled review and response.

Attachment handling introduces a second research thread. Rudd, Harang, and Saxe study malicious email attachment detection across heterogeneous file types and show that attachment-focused ML can be treated as a separate detection problem [6]. Zhang et al. show that MIME parsing ambiguity can let adversaries evade email attachment detectors, strengthening the case for preserving the raw message and separating MIME parsing from later detector outputs [7]. Cloud Email Analyzer follows this separation: the parser extracts evidence and file artifacts, the scanner evaluates each attachment, and the dashboard exposes per-file scan state instead of collapsing all file evidence into an opaque message label.

The workflow layer is related to serverless computing research. Serverless systems are commonly decomposed into small stateless functions that communicate through storage and cloud services, while workflow orchestration coordinates longer application logic [8]. Triggerflow and related work discuss state machines, DAGs, and trigger-based orchestration as patterns for composing event-driven functions [9]. Cloud Email Analyzer applies this pattern to email security: SES and Route53 provide ingress, EventBridge carries the normalized event, Step Functions coordinates analysis stages, S3 stores large artifacts, and DynamoDB stores queryable state. The positioning is summarized in Table 1.

Table 1

**Positioning against selected related work**

| Research line                 | Main emphasis in related work                                                                         | Cloud Email Analyzer emphasis                                                                     |
|-------------------------------|-------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Phishing ML detectors         | Supervised classification of phishing messages using engineered, text, URL, or embedding features.    | Uses a TF-IDF classifier as one detector inside a larger triage workflow.                         |
| Multi-module phishing systems | Combines several email components to improve phishing detection and reduce single-signal blind spots. | Combines text verdicts with attachment scanning, storage, authorization, and audit review.        |
| Attachment and MIME research  | Studies malicious attachments and evasion through parsing ambiguity.                                  | Preserves raw MIME, stores parsed artifacts, and exposes per-attachment scan evidence.            |
| Serverless workflow research  | Coordinates event-driven functions through state machines, triggers, and cloud services.              | Applies event-driven orchestration to email ingestion, analysis, verdict aggregation, and review. |

### 3 System Overview and Requirements

Cloud Email Analyzer is designed as an AWS email security inbox for organizations that need to inspect suspicious messages without losing the original evidence. The product flow is direct: receive an email, store the raw MIME object, extract structured evidence, run detectors, compute verdicts, and expose the result to the appropriate user or administrator. The reviewer should see

both the short verdict and the supporting evidence: sender and recipients, subject, parsed text and HTML, URLs and domains, attachment metadata, hashes, scan signatures, classifier confidence, processing timeline, and audit trail.

The AWS ingress layer uses Route53-hosted DNS/MX records and Amazon SES receipt rules. SES can receive mail for a verified domain and store accepted messages in S3 [10]. An ingestion adapter then publishes the canonical MailReceived event, after which the rest of the system is independent of SMTP details. This choice also supports development with LocalStack: the team can exercise S3, EventBridge, Step Functions, Lambda-style handlers, and DynamoDB during implementation while keeping the same event shape and storage boundaries used by the AWS design.

The system requirements can be grouped into six categories. First, raw-message preservation is required because later stages and reviewers may need to inspect or reprocess the original MIME object. Second, ingestion must be normalized so the analysis pipeline receives a stable event contract. Third, processing must be staged because recipient mapping, MIME parsing, ML classification, file scanning, and aggregation have different failure modes and outputs. Fourth, the database model must support both user inbox queries and security-administrator review. Fifth, back-end authorization must be authoritative because hiding navigation elements in the frontend is not a security control. Sixth, review must be auditable: security-relevant actions such as login, message view, reprocess, and attachment download should leave records.

Table 2

Cloud Email Analyzer components and responsibilities

| Component                       | Responsibility                                                                                                                  | Main output                                  |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|
| SES and Route53 ingress         | Receive mail with SES for a domain whose MX records are managed in Route53, then place the raw MIME object in cloud storage.    | Raw message object and envelope metadata.    |
| EventBridge boundary            | Carry the canonical MailReceived event into the analysis workflow.                                                              | Stable event contract.                       |
| Step Functions workflow         | Orchestrate the sequential stages with retry behavior.                                                                          | Ordered processing state.                    |
| Ledger and recipient stages     | Create the message row, map envelope recipients to owners, and write inbox rows.                                                | Message ownership and inbox visibility.      |
| Parser stage                    | Parse MIME headers, text, HTML, and attachments; store parsed JSON and file artifacts.                                          | Parsed body artifact and attachment records. |
| Detection stages                | Run packaged TF-IDF classifier inference and ClamAV attachment scanning.                                                        | ML verdicts and per-file scan evidence.      |
| Aggregation stage               | Combine ML and attachment evidence into message and inbox verdicts.                                                             | Final verdict: SAFE, SUSPICIOUS, or UNSAFE.  |
| FastAPI API and React dashboard | FastAPI enforces authorization and exposes message/admin endpoints; React presents the user and administrator review workflows. | Authenticated user/admin security inbox.     |

Table 2 summarizes the main components. The design is intentionally practical: small services perform bounded tasks, object storage carries large artifacts, and DynamoDB tables store queryable state for the API and dashboard. This division keeps large evidence objects outside query rows while preserving clear event-driven boundaries.

## 4 Event-Driven AWS Architecture

The architecture centers on a normalized event boundary. Amazon SES receives inbound mail for a domain whose MX records are managed in Route53 and stores the raw MIME message in S3. An ingestion adapter publishes a MailReceived event that contains the message identifier, tenant, received timestamp, raw object pointer, envelope sender, envelope recipients, header hints, and SES verdict metadata. The event carries metadata and pointers; it does not carry raw email bodies, HTML, or attachment bytes.

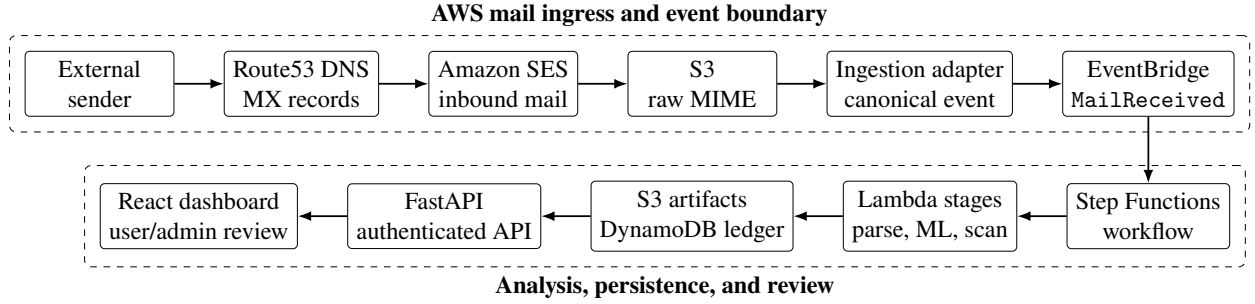


Fig. 1. AWS-oriented architecture of Cloud Email Analyzer from inbound mail to authenticated review.

Figure 1 shows the end-to-end cloud shape. EventBridge is used because event buses route events from sources to targets and rules evaluate event patterns before delivery [11]. Step Functions supplies the workflow layer; AWS describes a state machine as a workflow composed of event-driven steps, with task states representing units of work performed by AWS services or APIs [12]. S3 keeps large immutable artifacts, and DynamoDB keeps compact queryable state.

The canonical event follows EventBridge conventions with a source, a detail type, and a detail object. The detail contains identity, envelope, header, storage, and upstream-verdict metadata. The important architectural point is that later pipeline stages depend on the event contract, not on the exact mail-receipt mechanism. This keeps ingestion, processing, and review loosely coupled.

The implementation also defines deterministic identifiers. A stable message identifier can be derived from the SHA-256 digest of raw message bytes. Attachment identifiers combine an index, a SHA-256 prefix, and a sanitized filename. These identifiers are not proofs of safety, but they make artifact pointers and database rows predictable across repeated processing. That property matters for idempotency: duplicate event handling should update the same logical message rather than creating unrelated records.

LocalStack is used in development to run AWS-shaped services locally while validating the same boundaries: object storage, event routing, workflow orchestration, Lambda-style execution, and DynamoDB state. The operational architecture remains the AWS-oriented flow shown in Fig. 1.

## 5 Processing Pipeline and Data Model

The Step Functions workflow is sequential in the current design. Equation 1 gives the ordered execution chain:

$$\begin{aligned}
 \text{MailReceived} &\rightarrow \text{InitLedger} \rightarrow \text{ResolveRecipients} \rightarrow \text{ParseEmail} \\
 &\rightarrow \text{PhishingML} \rightarrow \text{AttachmentScan} \rightarrow \text{AggregateVerdicts}.
 \end{aligned} \tag{1}$$

Each task includes retry behavior, and the stages are written so repeated invocations update deterministic records rather than creating a new logical message each time. Figure 2 separates the

main control-flow chain from the persistence layer: parsed artifacts are written to S3, while final message state is written to DynamoDB. PhishingML reads parsed text, and AttachmentScan reads extracted attachment artifacts. The bidirectional boundary indicates that workflow stages both read persisted artifacts and write updated state. The design could later parallelize ML and attachment scanning because both depend on parsed artifacts, not on each other. The sequential flow is easier to observe, explain, and audit.

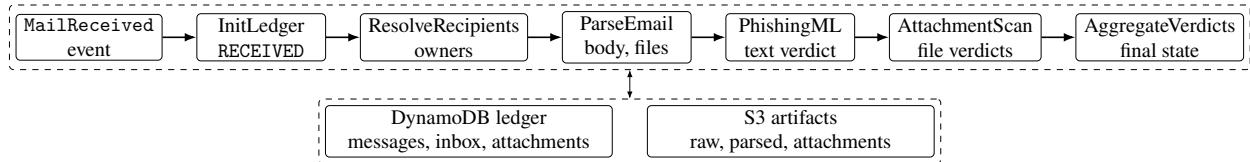


Fig. 2. Processing pipeline from normalized event to final message state.

## 5.1 Pipeline Stages

`InitLedger` creates the initial message ledger row. The row stores the tenant, message identifier, received timestamp, source, sender, recipients, subject, raw object pointer, status `RECEIVED`, and pending verdict fields. Conditional creation makes duplicate message events idempotent.

`ResolveRecipients` converts envelope recipients to ownership state. It lowercases each recipient, looks up the address in the mailbox ownership map, collects owner user identifiers, writes per-user inbox rows, and records unresolved recipients. Unknown recipients are assigned to administrator review. This design avoids relying only on visible `To` or `CC` headers, which may not match the SMTP envelope.

`ParseEmail` downloads the raw MIME object, extracts headers, plain text, HTML, and attachments, writes a parsed body artifact, writes extracted attachment bytes, creates attachment records, and marks the message as `PARSED`. If artifact writing fails, the message is marked as failed with a parse error.

`PhishingML` reads the parsed artifact, combines subject and body text, applies the same cleaning logic used by training, loads packaged model artifacts, and writes ML verdict, category, confidence, model version, model source, and scanned timestamp.

`AttachmentScan` reads extracted attachment pointers, downloads each file, recomputes SHA-256, writes the bytes to temporary storage, invokes `clamscan`, and updates each attachment with scan status, scan verdict, signature, hash, and timestamp. ClamAV documents `clamscan` as a command-line scanner for files and directories [13]. The EICAR anti-malware test file is used as a harmless standard detector trigger during validation [14].

`AggregateVerdicts` reads the message and all attachment records, computes an aggregate virus verdict, combines it with the ML verdict, updates the message row, and propagates final verdict fields to inbox rows. This last stage is important because the dashboard should show a compact user-facing verdict while preserving detailed detector outputs for review.

## 5.2 Data Model

DynamoDB is used as the application database. Its core model consists of tables, items, attributes, and primary keys [15], which fits the project because the system needs direct message lookups, user inbox queries, attachment queries by message, and audit records by tenant/time.

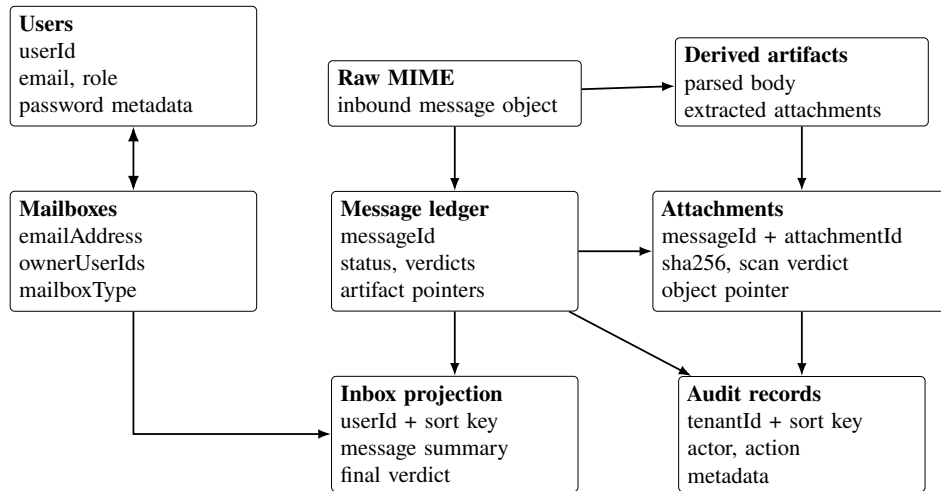


Fig. 3. Main persisted entities and artifact pointers used by the analysis workflow.

Figure 3 summarizes the stored entities. The message ledger is the central record. Inbox projections are optimized for user-specific inbox queries. Attachment records hold per-file metadata, storage pointers, hashes, and scan results. Users and mailboxes support authentication and recipient ownership. Audit records store security-relevant actions. Table 3 lists the same storage split by store, key shape, and purpose.

Table 3

| Persisted records and artifacts |                                       |                                                                                 |
|---------------------------------|---------------------------------------|---------------------------------------------------------------------------------|
| Store                           | Key or identifier                     | Purpose                                                                         |
| S3 raw email                    | Inbound message object key            | Preserves the original MIME message for inspection and reprocessing.            |
| S3 parsed body                  | Message-scoped parsed artifact key    | Stores parsed headers, summary, plain text, HTML, and attachment summaries.     |
| S3 attachments                  | Message-scoped attachment object keys | Stores extracted attachment bytes separately from workflow payloads.            |
| DynamoDB message ledger         | messageId                             | Central state for status, ownership, raw/artifact pointers, and verdicts.       |
| DynamoDB inbox projection       | userId, time-sort key                 | User-specific query rows with display status and verdicts.                      |
| DynamoDB attachment records     | messageId, attachmentId               | Per-file metadata, SHA-256, scanner status, verdict, and signature.             |
| DynamoDB audit records          | tenantId, time-sort key               | Records security-relevant actions such as login, view, reprocess, and download. |

The split between S3 artifacts and DynamoDB records is intentional. Raw MIME messages, parsed bodies, and attachment bytes are large or semi-structured objects and belong in object storage. DynamoDB stores compact state, ownership, verdicts, and pointers. Step Functions passes identifiers and S3 locations rather than embedding whole email bodies or files in workflow payloads. This keeps each stage focused and makes reprocessing possible from the raw pointer.

## 6 Classification, Attachment Scanning, and Verdict Aggregation

The detection layer has three parts: phishing text classification, attachment scanning, and final verdict aggregation. The project does not assume that a single detector is sufficient. Each

detector contributes evidence that is stored and then simplified into a message-level verdict for the dashboard.

## 6.1 Text Classification

The ML inference stage loads a packaged phishing model and a packaged TF-IDF vectorizer. The model-training pipeline represents each message with a TF-IDF vectorizer using 15,000 maximum features, word unigrams and bigrams, minimum document frequency of 10, and English stop-word removal. Four candidate classifiers were evaluated: logistic regression, multinomial Naive Bayes, random forest, and a linear SGD classifier. The selected packaged classifier is the SGD model with modified Huber loss and balanced class weights; the evaluation also records precision, recall, and F1 alongside validation accuracy.

The text normalization step lowercases input, replaces URL-like tokens, removes selected mail headers and corpus-specific tokens, replaces digits, removes most punctuation except common email characters, and collapses whitespace. The cleaned subject and body text are then transformed into TF-IDF features. Scikit-learn is the underlying ML library; its JMLR paper describes it as a Python module covering supervised and unsupervised learning algorithms with a consistent API [16]. Equation 2 gives a compact formulation of the representation:

$$x_{d,t} = \text{tf}(t, d) \cdot \log \left( \frac{1 + N}{1 + \text{df}(t)} \right), \quad (2)$$

where  $x_{d,t}$  is the feature value for term  $t$  in document  $d$ ,  $N$  is the number of training documents, and  $\text{df}(t)$  is the number of documents containing term  $t$ . The exact smoothing and normalization are delegated to the saved vectorizer parameters.

The inference output is an ML object with verdict, category, score stored in the confidence field, model version, model source, and feature version. In the current binary label mapping, class 1 maps to PHISHING with category `credential_theft`, while class 0 maps to LOW\_RISK with category `ham`. This mapping is an intermediate classifier interface: later versions can replace the hard class mapping with calibrated thresholds or additional categories. A TF-IDF linear model fits the workflow stage because it is fast, inspectable at the feature level, and supported by strong baselines in phishing email literature [1,4].

## 6.2 Attachment Scanning

The attachment scanner uses records produced by the parser. For each attachment, it reads the object, computes SHA-256, invokes `clamscan`, and maps scanner exit code 0 to `SAFE`, exit code 1 to `UNSAFE`, timeouts to `TIMEOUT`, and other unexpected results to `SCAN_ERROR`. The scanner also stores any ClamAV signature string returned by the command. The production engineering track includes a managed signature update lifecycle so scanner definitions remain fresh over time.

The project deliberately stores attachment scan data per file rather than only at the message level. This lets the dashboard show which file triggered an unsafe verdict, the filename, content type, size, SHA-256 hash, signature, scan status, and download action. It also allows the aggregate stage to recompute message-level virus state from child records.

## 6.3 Final Verdict Rules

Final verdict aggregation is intentionally simple. Let  $m$  be the ML verdict and  $v$  be the aggregate attachment verdict. The rule in Equation 3 captures the current implementation:

$$F(m, v) = \begin{cases} \text{UNSAFE}, & m = \text{PHISHING} \vee v = \text{UNSAFE}, \\ \text{SUSPICIOUS}, & m \in \{\text{ERROR}, \text{PENDING}\} \vee v = \text{PARTIAL}, \\ \text{SAFE}, & \text{otherwise.} \end{cases} \quad (3)$$

The aggregate virus verdict is SAFE when there are no attachments or all attachments are safe, UNSAFE when at least one attachment is unsafe, and PARTIAL when any scan remains pending or ends in an error-like state. This rule set is conservative: unavailable detector data does not become a clean result.

## 7 Dashboard and Review Workflow

The backend is a FastAPI application with authentication routes, message routes, admin routes, audit logic, and a Lambda-compatible handler. The frontend is a React application. The important security boundary is server-side authorization: frontend navigation changes the user experience, but the API decides which records a caller may read or modify.

Authentication currently uses application-managed HS256 JWTs backed by user records. Password verification uses PBKDF2-SHA256 with per-user salt and iteration metadata stored in the user record. After login, each JWT includes signed claims for the user identifier, login email, role, tenant, and expiry time; the API reads those claims before authorizing message and administration requests. The planned AWS identity track is Cognito migration so user and token management can be delegated to a managed identity service.

Normal users can list and view only messages mapped to their user identifier through inbox projections and owner identifiers. Administrators can access broader review endpoints. Message detail, indicators, timeline, attachment list, and attachment download repeat the same ownership-or-admin check. This repetition matters because direct API calls should not bypass frontend navigation.

The dashboard exposes several user-visible workflows. The inbox screen provides search, status filters, verdict filters, attachment filters, sorting, pagination, and compact message rows. The message detail screen loads the message, attachments, indicators, and workflow timeline. It displays threat summary, verdict cards, email body views, metadata, ML analysis, workflow timeline, threat evidence, URLs and domains, hashes, and attachment cards. Risky attachment downloads require explicit acknowledgement before the browser downloads the file. The API also logs safe and risky download actions.

Administrators receive additional review screens. The admin dashboard shows security totals, verdict counts, top senders, mailbox activity, trends, recent messages, audit events, users, and mailbox mappings. The admin message review page supports filtering and pagination over all messages. The audit log page supports filtering by action, actor, role, and message identifier. The reprocess endpoint can start a new workflow execution for a message with an available raw artifact pointer.

## 8 Evaluation Results

Figure 4 reports the internal validation confusion matrix: 7,748 true negatives, 158 false positives, 152 false negatives, and 7,649 true positives. The diagonal entries are the correctly classified messages, while the off-diagonal entries show the two error types. Computed from those counts, internal validation accuracy is 0.9803. The class-1 precision, recall, and F1 are approximately 0.9798, 0.9805, and 0.9801.

| Actual / Predicted | Low risk                       | Phishing                       |
|--------------------|--------------------------------|--------------------------------|
| Low risk           | <b>7,748</b><br>true negatives | <b>158</b><br>false positives  |
| Phishing           | <b>152</b><br>false negatives  | <b>7,649</b><br>true positives |

Fig. 4. Internal validation confusion matrix for the packaged phishing classifier.

The quantitative classifier evaluation compares logistic regression, multinomial Naive Bayes, random forest, and the SGD-based classifier used by inference. Figure 5 uses a 0.90–1.00 y-axis so the differences between high-scoring classifiers remain visible. It shows the SGD model as marginally above the other evaluated models on the internal validation split, with accuracy, precision, recall, and F1 around 0.980. Logistic regression is close behind, while Naive Bayes and random forest trail on the same metrics. These values describe the training and validation setting used for the packaged model, not live organizational traffic.

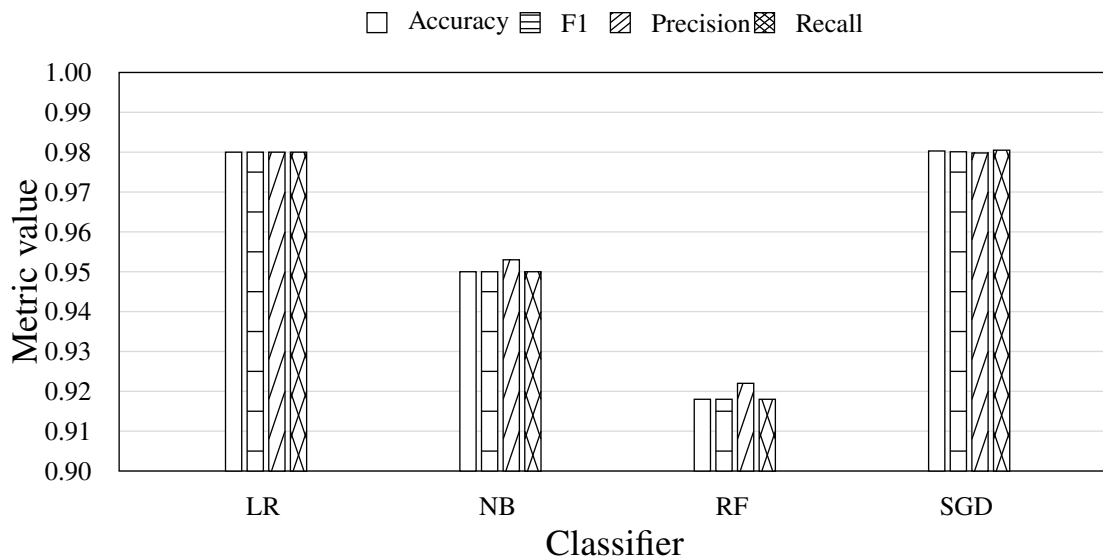


Fig. 5. Classifier comparison based on the ML evaluation results, with the y-axis focused on 0.90–1.00.

The training corpus combines CEAS\_08 as the largest source at 49.8%, followed by Enron at 37.7%, SpamAssassin at 7.4%, Ling at 3.6%, and Nazario at 1.6%. Enron is a well-known email corpus introduced for email classification research [17]; the other named sources are represented in curated phishing-email and spam-corpus distributions [18–20]. These results support the use of the packaged classifier inside the triage workflow while keeping the evaluation scope limited to the training and validation data.

Beyond the classifier evaluation, implementation tests cover the processing and review path. Unit tests cover parser behavior for body extraction, attachment extraction, artifact writing, and failure marking. API tests cover authorized and unauthorized attachment download, missing attachments, paginated listing, and audit-log failure handling. Ledger tests cover initial row creation and duplicate-message behavior. These checks exercise parser behavior, API authorization, ledger idempotency, and audit handling across the main workflow stages.

## 9 Limitations and Future Work

The first limitation is detector scope. The phishing classifier scores text, and ClamAV scans extracted files. Sender reputation, URL reputation, sandbox detonation, attachment content disarm, DKIM/SPF/DMARC policy reasoning, and organization-specific allow/block logic are outside the current detector set. The architecture leaves room for these signals because verdict aggregation already combines independent evidence streams.

The second limitation is operational policy. Attachment handling can be expanded with explicit size limits, archive-depth rules, retention policy, quarantine policy, and reviewer escalation rules. Raw HTML rendering must remain sanitized or isolated, and risky-download controls should be paired with organizational policy rather than treated as a complete safety boundary.

The third limitation is identity and scanner operations. The authentication layer is moving toward Cognito so identity, token handling, and user lifecycle can be managed as AWS services. The ClamAV stage needs a production signature-update lifecycle so scan definitions stay current across function/container lifetimes.

Future work follows from these limits. The workflow can add URL and domain reputation checks, header-authentication scoring, richer indicator extraction, and separate aggregation weights for phishing and malware evidence. The dashboard can add analyst notes, case assignment, exportable reports, and retention controls. The ML workflow can add threshold calibration and model cards so classifier changes are reviewable over time.

## 10 Conclusions

Cloud Email Analyzer presents an AWS-oriented architecture for turning inbound email into reviewable security evidence. The project receives MIME messages, stores original content, normalizes ingestion through MailReceived, resolves recipient ownership, parses body and attachments, runs text classification, scans files, aggregates verdicts, and exposes the results through an authenticated dashboard with user and administrator views.

The strongest idea is not a single detector. It is the separation between ingress, analysis, storage, verdict aggregation, and review. Raw artifacts remain available, parsed artifacts are stored separately, detector outputs are visible, attachment evidence is preserved per file, and user actions are audited. That gives the system a credible path from incoming mail to accountable security triage, while leaving space for stronger detectors, managed identity, and operational hardening.

## REFERENCES

- [1] I. Fette, N. Sadeh, and A. Tomasic, “Learning to detect phishing emails,” in *Proceedings of the 16th International Conference on World Wide Web*, pp. 649–656, 2007.
- [2] A. Almomani, B. B. Gupta, S. Atawneh, A. Meulenberg, and E. Almomani, “A survey of phishing email filtering techniques,” *IEEE Communications Surveys & Tutorials*, vol. 15, no. 4, pp. 2070–2090, 2013.
- [3] M. Khonji, Y. Iraqi, and A. Jones, “Phishing detection: A literature survey,” *IEEE Communications Surveys & Tutorials*, vol. 15, no. 4, pp. 2091–2121, 2013.
- [4] A. A. Tawil, L. Almazaydeh, D. Qawasmeh, B. Qawasmeh, M. Alshinwan, and K. Elleithy, “Comparative analysis of machine learning algorithms for email phishing detection using TF-IDF, Word2Vec, and BERT,” *Computers, Materials & Continua*, vol. 81, no. 2, pp. 3395–3412, 2024.

- [5] J. Lee, F. Tang, P. Ye, F. Abbasi, P. Hay, and D. M. Divakaran, “D-Fence: A flexible, efficient, and comprehensive phishing email detection system,” in *2021 IEEE European Symposium on Security and Privacy*, pp. 578–597, 2021.
- [6] E. M. Rudd, R. Harang, and J. Saxe, “MEADE: Towards a malicious email attachment detection engine,” in *2018 IEEE International Symposium on Technologies for Homeland Security*, 2018.
- [7] J. Zhang, J. Chen, Q. Wang, H. Zhang, C. Wang, J. Zhuge, and H. Duan, “Inbox invasion: Exploiting MIME ambiguities to evade email attachment detectors,” in *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*, pp. 4066–4080, 2024.
- [8] E. Marin, D. Perino, and R. D. Pietro, “Serverless computing: A security perspective,” *Journal of Cloud Computing*, vol. 11, no. 69, 2022.
- [9] A. Arjona, P. G. López, J. Sampé, A. Slominski, and L. Villard, “Triggerflow: Trigger-based orchestration of serverless workflows,” *Future Generation Computer Systems*, vol. 124, pp. 215–229, 2021.
- [10] Amazon Web Services, “Amazon SES email receiving concepts and use cases.” <https://docs.aws.amazon.com/ses/latest/dg/receiving-email-concepts.html>. Accessed 2026-05-09.
- [11] Amazon Web Services, “Event buses in amazon eventbridge.” <https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-event-bus.html>. Accessed 2026-05-09.
- [12] Amazon Web Services, “Learn about state machines in step functions.” <https://docs.aws.amazon.com/step-functions/latest/dg/concepts-statemachines.html>. Accessed 2026-05-09.
- [13] ClamAV, “Clamav documentation: Scanning.” <https://docs.clamav.net/manual/Usage/Scanning.html>. Accessed 2026-05-09.
- [14] EICAR, “Download anti malware testfile.” <https://www.eicar.org/download-anti-malware-testfile/>. Accessed 2026-05-09.
- [15] Amazon Web Services, “Core components of amazon dynamodb.” <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.CoreComponents.html>. Accessed 2026-05-09.
- [16] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, “Scikit-learn: Machine learning in python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [17] B. Klimt and Y. Yang, “The Enron corpus: A new dataset for email classification research,” in *Machine Learning: ECML 2004*, vol. 3201 of *Lecture Notes in Computer Science*, pp. 217–226, Springer, 2004.

- [18] A. I. Champa, M. F. Rabbi, and M. F. Zibran, “Curated datasets and feature analysis for phishing email detection with machine learning,” in *2024 3rd IEEE International Conference on Computing and Machine Intelligence*, pp. 1–7, 2024.
- [19] Apache SpamAssassin Project, “Spamassassin public mail corpus.”  
<https://spamassassin.apache.org/old/publiccorpus/>. Accessed 2026-05-09.
- [20] CLAIR Collection, “Fraudulent e-mail corpus.”  
<https://www.kaggle.com/datasets/rtatman/fraudulent-email-corpus>. Accessed 2026-05-09.